

Colour themes in PDF: a proposal for dark mode and accessibility

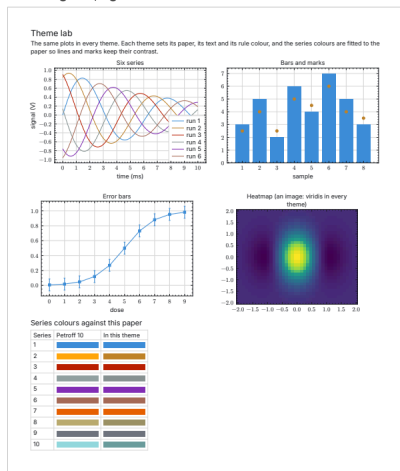
Mathew Newman, October 2026

PDF readers make their own dark mode by inverting or recolouring the page, which spoils colour designs and charts. This proposes a way to put checked themes inside the PDF, such as dark, tinted or higher contrast, designed by the author or worked out by the tool, so a reader can switch between them without storing the document twice.

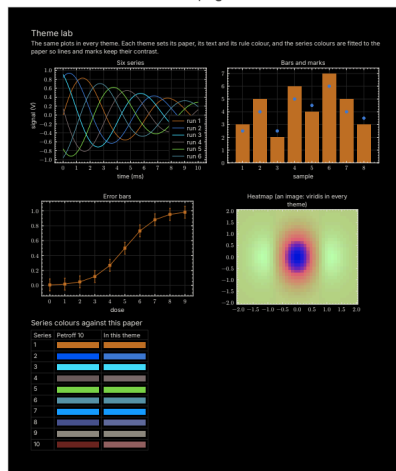
The problem

Readers that offer a dark mode make it up themselves. They invert the page or swap its colours for their own, because the PDF never says what each colour is for. That works for black text on white, but it spoils anything designed in colour: brand colours change hue, charts lose their key and photos turn into negatives.

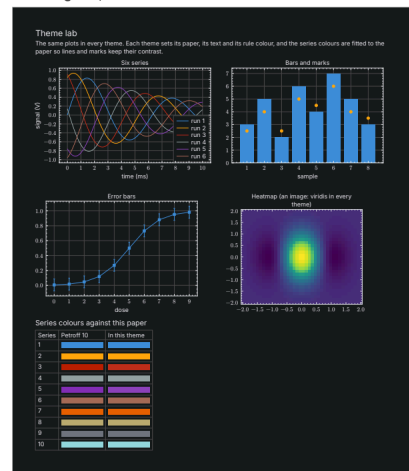
As designed, light



As a reader that inverts the page shows it



As designed, dark



Readers are already working round this. Firefox's PDF viewer can force its own page colours ([pdf.js issue 18680](#)), and people have asked for reader themes in [Zotero](#) and [SumatraPDF](#). Browser extensions such as Dark Reader can't reach PDFs in Firefox at all ([discussion](#)).

An author who wants a proper dark version has two options today, and both cost something:

- Two files. Readers have to pick the right one, and every change is made twice.
- Two layers in one file (optional content). Only some readers show a switch, and the text and drawings are stored twice. In my test on a page of plots, PDFium, the engine in Chrome, returned the text twice, while Poppler and MuPDF skipped the hidden copy. The file grows by 41% this way.

What already works elsewhere

Fonts, web pages and app frameworks all let the author ship their own dark version. PDF is the odd one out.

Format	Where the author puts the dark colours	What picks them	Source
OpenType colour fonts	Extra colour palettes, each flagged as usable on a light or a dark background	The app, from the background it draws on	CPAL table
Web pages	Style rules for prefers-color-scheme: dark, and dark images as <picture> sources	The browser, from the system or browser setting	prefers-color-scheme , picture
WinUI apps	Light and Dark theme dictionaries of named brushes	Windows, from the app or system theme	XAML theme resources
PDF	Nowhere	The reader guesses	

Colour fonts are the closest match to what PDF needs: the same shapes drawn with a second palette, picked by the background.

Earlier work

I looked for the same idea in PDF and didn't find it written down.

- On the [open request for dark mode in PDF](#), the chair of the PDF TWG says BenQ presented a somewhat similar idea at the PDF Association's [imaging model symposium](#) in October 2023. The symposium ran under the Chatham House Rule, and I couldn't find the talk or anything BenQ has published on it.
- The dark mode patents I found work the colours out when the content is shown, which is what readers do today. Microsoft's, for mail, parses the content and adjusts any colour that falls under a contrast threshold ([US 11,107,258](#)). None of them store an author's alternate colours in the file.
- The closest stored palette is an Adobe patent from 2003 on drawing a document with reduced palettes for small screens, with the views kept in the file. It expired in 2024 ([US 7,324,119](#)).
- The LaTeX [darkmode package](#) gives each colour a light and a dark value in the source, but it still writes one PDF per theme.

This was a search of Google Patents and the web in October 2026. It isn't a legal opinion on what a patent covers.

Goals

A theme has to look right, cost little and break nothing that works today. It should:

- Store the content once. Text, structure and drawing order are shared, so search, copy and paste, and screen readers all see one document.
- Cost bytes per colour, not per page. A dark theme for a text document should add about as much as a colour palette.
- Be checked by the author. The author picks the other colours and checks their contrast when the file is made, and the file can say what they were checked against.
- Cover everything that carries colour: text, lines, fills, gradients, images, annotations, form fields, bookmarks and tags. A chart's key has to keep matching its lines.
- Follow the person's setting. A reader picks the theme that matches the system's light or dark mode, and offers a switch.
- Be safe in older readers. A reader that knows nothing about themes shows the default colours, exactly as today.
- Allow more than light and dark. The same mechanism should carry a warm or cream theme, a higher contrast theme, or chart colours that work for colour-blind readers.
- Work with PDF/UA-2, PDF/A and signatures. Choosing a theme changes nothing in the file, so tags, archive rules and signatures still hold.

Who themes help

Themes are an accessibility feature as much as a comfort one. The research supports giving people a choice of checked themes, not a single dark mode.

Who	What the research found	What a theme gives them
People with dyslexia, and other readers, on screen	Warm backgrounds (peach, orange, yellow) gave faster reading than cool ones, in 341 people of whom 89 had dyslexia (Rello & Bigham 2017)	A warm or cream theme, as the British Dyslexia Association's style guide recommends
Readers who differ in what works	In 30 children, 15 of them with dyslexia, the effect of the background colour varied a lot from child to child (Vajs et al. 2022). In a survey of 204 neurodivergent adults, background colour was the one setting each group preferred differently (Otermans et al. 2025)	A choice of themes, each one checked
People with low vision	Of 19 adults, five read light text on dark faster, and four of the five had cloudy eye media (Rubin & Legge	Dark and higher contrast themes, picked per person

Who	What the research found	What a theme gives them
	1989). In 48 children it helped some, and no test predicted which (Loh et al. 2024)	
Older readers	Dark isn't better for everyone. Light text on dark took older adults more effort in a bright room (Sethi & Ziat 2023)	Light stays the default, and dark is a choice
People with light sensitivity or migraine	18 people with migraine with aura did a word search 46% faster under light of a colour they chose, given through tinted lenses (Vieira et al. 2020)	Darker or tinted paper without losing the design
Anyone reading charts	In 134 people, whether light or dark gave better chart reading varied by person, and often wasn't the one they preferred, so the authors recommend offering both (While & Sarvghad 2024)	Charts whose colours are checked in each theme

None of these studies used themed PDFs, and most are small. They tested colour on screens, on paper and through lenses. What they agree on is that the colours that work best differ from person to person. Colour isn't a treatment for dyslexia itself. A systematic review of 51 studies found coloured overlays and lenses can't be endorsed for reading difficulties, and put the benefits people report down to placebo and practice ([Griffiths et al. 2016](#)). A second review found that filters chosen for the person improved reading in people with visual stress, which it puts at about one in five dyslexic readers, and it also says they don't treat dyslexia ([Evans & Allen 2016](#)). The two reviews disagree, and both ask for better trials. The case for themes is comfort and choice. [WCAG 2.2 criterion 1.4.8](#) asks that people can select text and background colours, and a 2012 study found PDF readers fell short of this ([Henry 2012](#)). An overlay or a reader's recolouring tints everything, charts included. A checked theme changes the colours without breaking what they mean.

Standard themes

The standard themes are built in from the start. The author makes the document once in each theme, from one source, with its colours, charts and pictures set for that theme. The tool that makes the file then draws each theme and measures every run of text against what is behind it. A theme that passes is marked as checked. One that fails is still written, without the mark, and the author is told which text fell short. The tool doesn't change a colour the author chose. A file doesn't have to carry all eight, and it can carry others.

Theme	Labels in the file	Text contrast it must reach
Light	ColorScheme Light	4.5:1 (WCAG 1.4.3, AA)
Dark	ColorScheme Dark	4.5:1
Light, more contrast	ColorScheme Light, Contrast More	7:1 (WCAG 1.4.6, AAA)
Dark, more contrast	ColorScheme Dark, Contrast More	7:1
Cream, Peach, Yellow, Turquoise	ColorScheme Light, Tint	4.5:1

The labels let a reader follow the system setting and keep a person's choice from one document to the next, so someone who reads best on peach paper picks it once. The tints follow the research above: warm colours read fastest in the largest study, but people differ, so there's a range rather than one.

A PDF with no source to rebuild can still be given themes. A tool works them out from the file's own colours, checks each one and leaves out any that fail. That is the fallback and it is less reliable: it worked on my short test documents and failed on a 31-page file that holds every kind of PDF object.

The proposal

Organise the file so that every colour is an entry in a palette, and let a theme swap the palette.

- Every solid colour is painted through a palette, which PDF already has as an Indexed colour space. The page says "paint with entry 0" rather than "paint #C0832A". PDF allows this for text and shapes as well as images, and every reader I tested draws it correctly.

- A theme lists objects to swap: each palette for its own version, plus any gradient, opacity setting, image or colour value that also differs, such as an annotation's colour. Objects are swapped by reference, so no page content changes.
- Each theme says whether it is light or dark, whether it has more contrast or a tint, which paper colour to draw on, and what its colours were checked against.
- The colours as written are the default theme. A reader that doesn't know about themes shows them, so the file still works everywhere.



Figure 1: How a reader applies a theme: one lookup changes.

The page always says `/Th25 CS 0 SC`. The theme only changes which palette `/Th25` points to.

Swapping objects rather than rewriting pages is what keeps the text, the tags and every shape the same in every theme. A theme can repaint the page, but it can't move, add or remove anything on it.

Only the author can make these choices. On my test page, headings and body text share one near-black, and in the dark theme they split into two greys. On the page of plots, a line and the swatch that shows its unadjusted colour are the same red in the light theme. In the dark theme the line's red moves towards the text colour to keep its contrast against the paper, and the swatch stays as it was. A reader that only sees the colours can't tell any of them apart.

Example

This is the theme dictionary from the themed copy of the page of plots, shortened. The prototype keeps it under a private key, `/XXThemes`. A standard would use a key such as `/Themes`.

```
% In the document catalog
/Themes <<
  /Default << /Type /Theme /Name (Light) /ColorScheme /Light /Paper [1 1 1]
    /Checked << /Standard (WCAG 2.2) /Criterion (1.4.3 Contrast (Minimum)) /Level /AA >> >>
  /Alternates [
    << /Type /Theme /Name (Dark) /ColorScheme /Dark /Paper [0.0941 0.098 0.1098]
      /Checked << /Standard (WCAG 2.2) /Criterion (1.4.3 Contrast (Minimum)) /Level /AA >>
      /Replace [ 94 0 R 160 0 R      % the text colour's palette
                99 0 R 177 0 R      % the plot colours' palette
                985 0 R 150 0 R     % the table's border colour in the tags
                ... ]              % and 3 more palettes
    >>
    << /Type /Theme /Name (Cream) /ColorScheme /Light /Tint /Cream /Paper [0.9922 0.9647 0.8902]
      /Checked << ... >>
      /Replace [ 99 0 R 186 0 R ... ]
    >>
    % then Light, more contrast; Dark, more contrast; Peach; Yellow; Turquoise
  ]
>>

% One of the palettes the page uses, /Th25, and two of the versions themes swap in
99 0 obj [/Indexed [/ICCBased 987 0 R] 5 <C0832A 879594 C0832A 879594 9F9464 6C9C9F>] endobj
177 0 obj [/Indexed [/ICCBased 987 0 R] 5 <FFA90E 94A4A2 FFA90E 94A4A2 B9AC70 92DADD>] endobj
186 0 obj [/Indexed [/ICCBased 987 0 R] 5 <B47C2C 808E8D B47C2C 808E8D 968D60 669295>] endobj

% Page content: the same bytes in every theme
/Th25 CS 0 SC      % a plot line: #C0832A by default, #FFA90E in the dark theme
```

Key	Value	Meaning
Name	Text string	The name the reader shows in its theme switch
ColorScheme	Light or Dark	Matched to the system or app setting, like CSS <code>prefers-color-scheme</code>
Contrast	More (optional)	A higher contrast theme, matched like CSS <code>prefers-contrast: more</code>

Key	Value	Meaning
Tint	A name (optional)	The tint of a tinted theme, such as Cream, so a reader can keep a person's choice across documents
Paper	Three numbers, sRGB	The colour the reader paints before drawing each page
Checked	Dictionary (optional)	What the author checked the theme against, and when
Replace	Pairs of indirect references	While the theme is shown, the first object of each pair is drawn as the second. Alternate themes only

The rules:

- Replace may name colour spaces, functions, shadings, patterns, graphics states, images and colour values outside the page, such as an annotation's colour. It never names a page, a content stream, a form or a font, so text and shapes can't change.
- A replacement has the same type as the object it replaces. An image keeps its width, height and soft mask, and a graphics state may change only opacity and blend mode.
- An Indexed palette holds at most 256 colours, so a document that needs more uses several palettes.

Every place a PDF keeps colour has a way in, so the same mechanism works for any file, not only mine:

Where colour lives	How a theme changes it
Solid colours in page content, in any colour space	Palette entries
Colours inside forms, tiling patterns, Type 3 glyphs, annotation appearances and soft masks	Palette entries, the same way
Gradients	A replacement gradient
Opacity and blend modes	A replacement graphics state
Images	A replacement palette, or a replacement image
Stencil masks	The fill colour's palette entry
Inline images	Written as ordinary images first
Annotation, form field, bookmark, structure and portfolio colours	Replacement colour values
The page background	Paper

Authoring

Tools that already name their colours can write themes directly. Others can build the document once per theme and merge the builds.

- Tools that name colours, such as Office theme colours, CSS custom properties, design tokens or the colour variables in a Typst source, can write each named colour as a palette entry and each theme as a lookup table.
- Tools that don't can build once per theme and merge, as the prototype does. The builds must draw the same shapes and the same text, and only the paint may differ. That can take small changes to the source. Where one theme fills something with a gradient, the others need a gradient too, even one with every stop the same colour, and an outline drawn in one theme has to be drawn in the others, fully transparent where it shouldn't show. My page of plots needed one change: it printed its theme's name and contrast figures on the page, and those had to go.
- In Typst every theme's build already comes from one source, so the compiler could write the themes on export.
- A PDF with no source to rebuild can still get themes. The tool works them out from the file's colours: light surfaces take the tint or go dark, text moves in lightness until it reaches its target, and in a dark theme a highlighter's Multiply blend becomes Screen.
- A chart's key and its lines are drawn from the same colour, so they stay matched in every theme.
- When a chart library draws an image twice with the same shapes, the merge keeps one plane of pixels and gives each theme a palette. A photo usually stays as it is. If it must change, the theme swaps in a second image and pays for its bytes.
- The author gets a quick switch between the themes in the tool that makes the PDF, like a print preview, so each view can be looked over before the file goes out. The contrast check catches faint text, but not a logo that looks wrong on dark paper.
- Contrast is checked when the file is made, by drawing each theme and measuring every run of text against what is drawn behind it, with the WCAG 2.2 contrast ratio. A theme gets /Checked only if every

run passes. One the author built that fails is still written without it, and a worked-out theme that fails is left out.

- Lines and shapes are measured too, each against the colour drawn beside it, and the ones under 3:1 are listed for the author ([WCAG 1.4.11](#)). They are only listed, because the tool can't tell a plot line from a grid that is meant to be faint. On the page of plots it found orange marks drawn over blue bars at 1.1:1, which I'd missed.

What readers do

A reader picks a theme, paints the paper and swaps a few objects. Everything else is drawn as it is now.

- Treat the theme as a view setting, like draft mode or overprint preview. It changes how the pages are drawn, never the file, so there's nothing to save.
- Pick the theme that matches the app or system setting: light or dark, and more contrast when that's turned on. Remember a person's tint from one document to the next. If the document has no match, show the default, and recolour it the way readers do today if the person asks for that.
- Show a switch whenever a document has themes, for example View, Theme, then Light, Dark or Cream.
- Paint the theme's paper colour before drawing each page, and in thumbnails too.
- Swap each pair in Replace when objects are looked up. In the prototype that step is about 15 lines of Python, and 10 of JavaScript in the live demo.
- Colour-manage theme colours like any others. A theme says what each colour is, in sRGB or the document's own ICC space, and the screen's profile does the rest, as it does for the default theme.
- Print the default theme unless the person picks another one.
- Put the person's own settings first. Forced colours, such as a Windows contrast theme or Acrobat's [Replace Document Colors](#) preference, still win when they are turned on.
- Readers that don't know about themes ignore the key and draw the default. In the prototype, three PDF engines drew the themed file pixel for pixel like the original light build.

Accessibility, archiving, signatures and size

Themes change no text or structure, so tagged PDF, PDF/UA-2 and search work as before. Signatures need one new check, and the bytes added are small.

- Accessibility. The structure tree, the tags and the text are the same in every theme. A theme can also carry a higher contrast version the author has checked, so a PDF/UA-2 document can offer one without a second file. The PDF of this write-up is tagged, and veraPDF 1.28 passes it as PDF/UA-1 before the themes go in, after, and with each theme shown.
- Archiving. The default theme is the appearance an archive keeps. Alternate palettes are ordinary colour spaces built on an ICC profile, so a validator can check them like any other colour. veraPDF passes the themed page of plots as PDF/A-2b, PDF/A-2u and PDF/A-4, and this write-up as PDF/A-2a, so today's rules don't trip on the extra key. It only checks what the pages reach, though, and nothing reaches an alternate palette until its theme is shown. I validated each theme as a file of its own with its objects swapped in, and they all pass. A standard should say that replacement objects are held to the same rules as the objects they replace.
- Signatures. Choosing a theme changes no bytes, so a signature stays valid and covers every theme. The risk runs the other way: a theme could paint a line of text in the paper colour, or swap in an image that shows something else. That is close to the [shadow attacks](#) on signed PDFs. Because a theme can't touch content streams, the defence is narrow: a validator draws each theme and checks the contrast of its text, and a signing profile could allow image palettes but not whole replacement images.
- Size. A theme costs a few bytes per colour. Today's two-layer method stores the text and drawings twice; it only shares fonts and images.

Document	Light build	Themed, Light and Dark	Themed, all 8 themes	Two layers, today	Two files
A page of plots, built once in each of the 8 themes	38,318 B	39,034 B (+1.9%)	39,966 B (+4.3%)	54,117 B (+41%)	76,605 B
This write-up as a PDF, tagged, built once in each of the 8 themes	725 kB	725 kB (−0.1%)	726 kB (+0.1%)	not measured	1,451 kB

Document	Light build	Themed, Light and Dark	Themed, all 8 themes	Two layers, today	Two files
Test page: a vector chart, an image chart and a full-colour image	187,772 B	186,171 B (-0.9%)	186,590 B (-0.6%)	192,923 B (+2.7%)	376,732 B
Test document, 6 pages, every way a PDF carries colour	135,141 B	186,481 B (+38%)	198,559 B (+47%)	193,598 B (+43%)	268,814 B

Every file was saved the same way, with compressed object streams, so the sizes compare fairly. The page of plots has all eight themes built from its source, and as eight separate files it comes to 304,725 bytes. The write-up's seven extra themes cost about 500 bytes between them, and as eight separate files it comes to 5.8 MB. Most of its size is two pictures, which every theme shares. In the other two documents light and dark were built and the tool worked out the other six. The test page's themed file is smaller than its light build because the chart image is stored as palette indices rather than full colour. The test document is built to be hard: two of its images change completely between builds, so it has to store both versions.

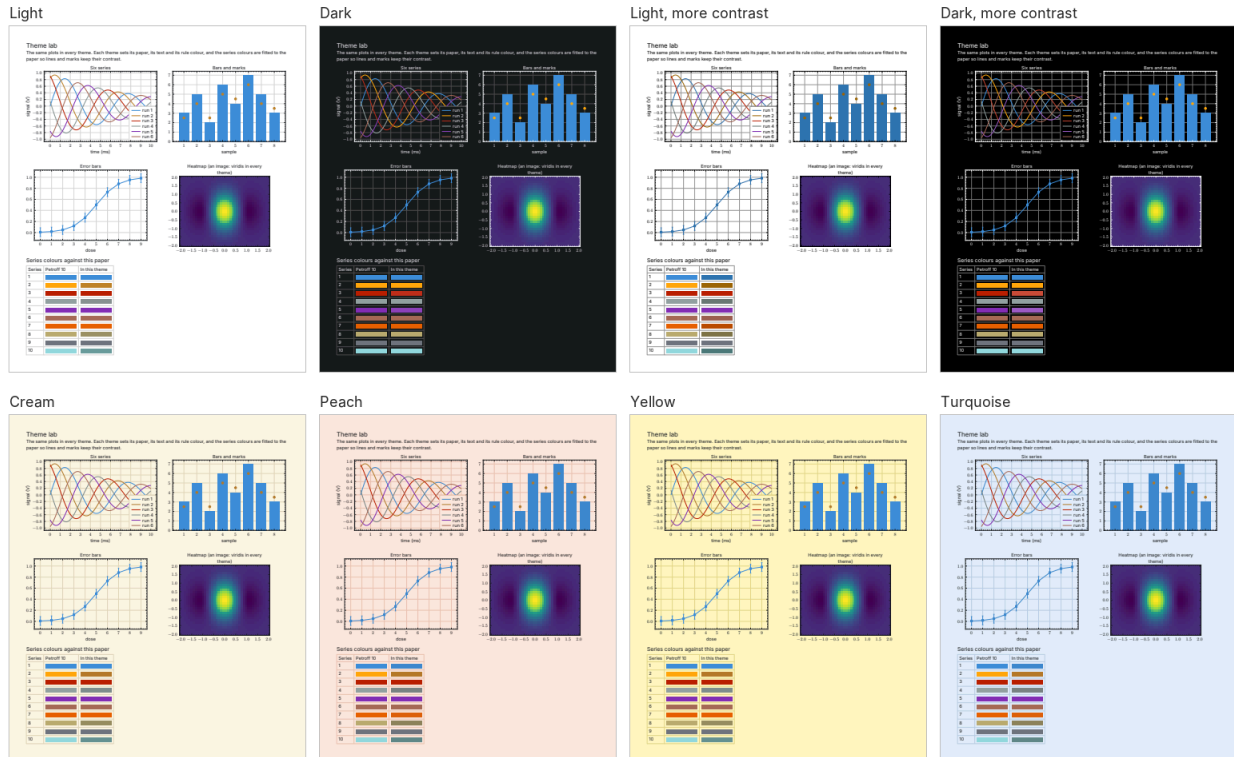
Prototype

One themed file drew pixel for pixel like the separate builds, in three PDF engines, for a page of plots built in eight themes, a test page and a six-page test document that uses every way a PDF can carry colour. Every one of the eight standard themes passed its contrast check in all three.

The PDF of this write-up is a themed file too. It is built once in each of the eight themes from one Typst source and merged. Each theme draws pixel for pixel like its own build in Poppler, MuPDF and PDFium, the text and the tagged content are the same in every theme, and the two plots further down are drawn in whichever theme the file is showing.

The prototype is about 3,800 lines of Python, using pikepdf to read and write PDF and PDFium to check contrast:

- A merge step takes one build per theme, or a single PDF, and writes one themed file. Every colour goes into a palette, whether it's in page content, a form, a pattern, a Type 3 glyph or an annotation's appearance. Each gradient, opacity, image or colour value that differs goes into that theme's Replace list.
- A derive step, for a document with no source to build from, works out the standard themes, draws each one, measures its text, fixes colours that fall short and leaves out any theme that still fails.
- A reader step shows a theme by swapping its objects, then draws the file as normal.
- The checks render every page at 144 dpi in Poppler 24.02, MuPDF 1.28.2 and PDFium 153 (the engine in Chrome), and compare text, tags, colour values, contrast and qpdf's structure check. The page of plots was checked later, in Poppler 25.07, MuPDF 1.28.2 and PDFium 156.



The page of plots from one file, in all eight themes. Each theme is built from the source, which sets the paper, the text and the rule colour and fits the series colours to the paper, so lines and marks keep at least 3:1 against it, and 4.5:1 in the two themes with more contrast. The heatmap is an image and keeps its colours in every theme.

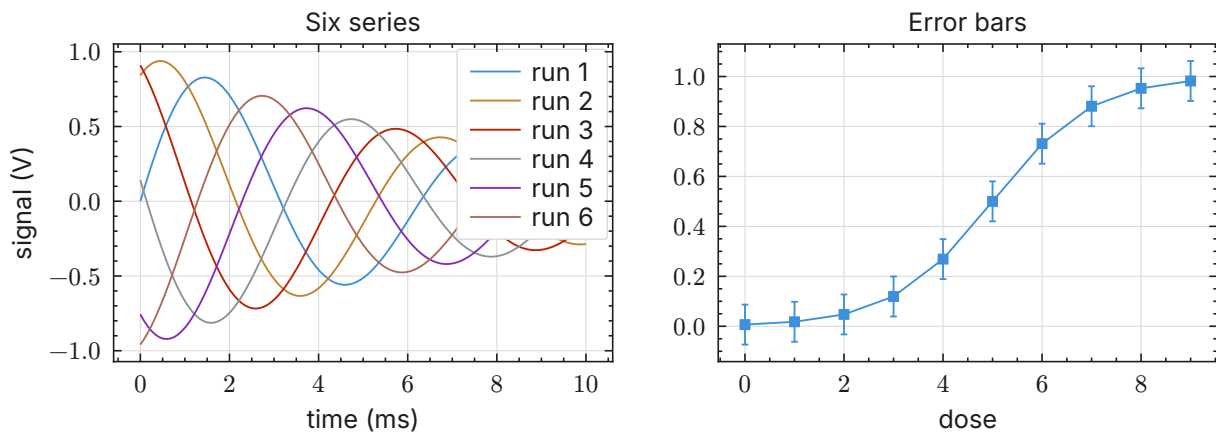


Figure 2: Two of the plots, drawn here in whichever theme this file is showing.

Check	Page of plots, 8 builds	Test page	Test document, 6 pages
Palette entries	14 for light and dark, 80 with all eight themes	8, plus 17 in the image chart	271
Objects the dark theme replaces	6: 5 palettes and a border colour in the tags	2: the palette and the image's palette	64, of every kind in the table under Example
Default theme against the light build, 3 engines	Every pixel identical	Every pixel identical	Every pixel identical, apart from PDFium's own bug on one swatch
Dark theme against the dark build, 3 engines	Every pixel identical, and the same for the other six themes	Every pixel identical	The same
Colour values outside the page	1 of 1 the same, in every theme	None in the file	42 of 42 the same
Text extracted	Identical	Identical	Identical
Tagged content kept	189 of 189	13 of 13	3 of 3

Check	Page of plots, 8 builds	Test page	Test document, 6 pages
Lowest text contrast, light and dark	15.34:1 and 13.2:1	5.07:1 and 6.63:1	5.67:1 and 6.16:1
Standard themes that passed	8 of 8	8 of 8	8 of 8
qpdf structure check	No errors	No errors	No errors

WCAG 2.2 asks for at least 4.5:1 for body text and 3:1 for large text, so every theme passes in all three documents, and the more contrast themes reach 7:1. The image chart's palette version took 1,034 bytes against 2,905 for the full-colour original.

PDFium's bug is with a colour space set without a colour: the specification says the space's starting colour applies, and PDFium keeps the previous colour instead. The themed file writes the colour out, so PDFium then draws it like the other two. The prototype's limits: worked-out themes treat every RGB ICC profile as sRGB, so they'd lose the most saturated colours of a Display P3 file; a gradient or image that differs between builds is swapped whole; and from a single PDF, only images that look like drawings are recoloured, never photos. You can switch themes in the [live demo](#).

Open questions for the working group

1. Where should themes live: a new catalog key, or a new kind of optional content configuration?
2. Should a theme be allowed to change line width through a graphics state, so a higher contrast theme can thicken thin lines?
3. Should the paper colour be an sRGB value or a palette entry, and should it also colour the reader's background around the pages?
4. Which theme prints when the default theme is dark?
5. How should a theme record what it was checked against: free text, as in the prototype, or a minimum contrast ratio that a validator can test? Should the check ask for a margin above the minimum, since screens and room light vary?
6. Should signature validation report the theme in use, and should validators draw every theme?
7. Should readers theme the appearances they make themselves, such as highlight colours and form fields without appearance streams?
8. Should HTML derived from tagged PDF carry the themes as CSS for `prefers-color-scheme`?
9. How does a reader choose between two themes for the same setting, such as Dark and Dark with more contrast?
10. Is the standard set right: Light, Dark, two more contrast themes and four tints? Should the tints be a fixed list, or any named tint?
11. For wide-gamut colours such as Display P3, should a writer work out themes in the document's own colour space rather than sRGB?
12. Dark paper on OLED screens: pure black can smear when scrolling, which is why many dark designs use a dark grey. Should Paper have a preferred range?
13. When a person adds a highlight while a theme is shown, which theme's colours does it take?
14. Should the check cover lines and shapes as well as text (WCAG 1.4.11), when a tool can't tell which of them carry meaning?
15. Should PDF/A and PDF/UA say that a theme's replacement objects must meet the same rules as the objects they replace, so a validator has to look at every theme?

Getting it adopted

A themed file already works in every reader, which shows its default colours. So tools can start writing themes now, and each reader that adds support improves every themed file already out there.

1. Write it up as a PDF Association application note or RFC through the [PDF TWG](#). It meets on the first and third Thursday of the month, 22:00 to 23:00 CET, and works on GitHub between meetings.
2. In parallel, propose an ISO technical specification. New features for PDF 2.0 arrive this way, like [ISO/TS 32001 to 32004](#), declared under the `iso_` prefix ([how](#)). In the UK, BSI's [IDT/1 committee](#) is the way into that work.
3. Until a standard name exists, register a developer prefix in [Adobe's PDF name registry](#), so early files from different tools work together.

4. Get it into the engines most people read with. pdf.js comes first: it's open source, used by Firefox and many web apps, and already recolours pages in high contrast mode. Then PDFium (Chrome, Android), Poppler and MuPDF. Apple's PDFKit and Adobe's engine, which [Edge now uses](#), are best reached through the PDF Association and ISO.
5. Get themed files made with no extra work for authors, with the standard themes and their checks built into the tools rather than offered as an option. Typst could write themes on export and put the switch in the previews of its [web app](#) and of the VS Code extension [Tinymist](#). Both can only invert the page's colours today, and the web app's invert turns red into green. Browsers could save web pages as themed PDFs, since pages already carry dark styles. The LaTeX [darkmode package](#) already gives each colour two values, and chart libraries have dark styles. A separate tool can add checked themes to any existing PDF, as the prototype's add command does.
6. Give people something to try: the live demo, the open-source prototype with its test files, a talk at PDF Days Europe and a write-up for pdfa.org.
7. Lead with accessibility ([Who themes help](#)). It's the argument that gets standards bodies and the public sector to act.

Sources

- [pdf.js issue 18680](#), [Zotero issue 4949](#), [SumatraPDF issue 743](#) and [Dark Reader discussion 7154](#): how readers deal with dark mode today
- [Changing background colours in Adobe Reader](#), University of Nottingham: Acrobat's Replace Document Colors
- [OpenType CPAL table](#), Microsoft: palettes for light and dark backgrounds
- [prefers-color-scheme](#), [prefers-contrast](#) and [picture](#), MDN
- [XAML theme resources](#), Microsoft
- [WCAG 2.2](#), W3C: contrast minimum and the contrast ratio
- [Understanding Success Criterion 1.4.11: Non-text Contrast](#), W3C
- [veraPDF](#), the open-source PDF/A and PDF/UA validator, version 1.28.2
- [pdf-issues 369](#), PDF Association: the open request for dark mode in PDF
- [US 11,107,258](#), Microsoft, and [US 7,324,119](#), Adobe: the closest patents I found
- [darkmode](#), a LaTeX package
- [Shadow attacks on signed PDFs](#), PDF Insecurity
- [ISO 32000-2:2020 Annex E, Extending PDF](#), PDF Association: second- and third-class names
- [PDF name registry](#), Adobe
- [Defining extensions to PDF](#), ISO TC 171
- [Current ISO work related to PDF](#) and [PDF TWG](#), PDF Association
- [Typst web app issue 307](#) and [Tinymist's preview](#): how Typst's previews handle dark mode today

Research on colour, contrast and reading:

- [Good Background Colors for Readers: A Study of People with and without Dyslexia](#), Rello and Bigham, ASSETS 2017
- [Spatiotemporal Eye-Tracking Feature Set for Improved Recognition of Dyslexic Reading Patterns in Children](#), Vajs et al., Sensors 2022
- [How do adults with neurodevelopmental disorders prefer information being presented?](#), Otermans et al., Educational Media International 2025
- [Psychophysics of Reading VI: The Role of Contrast in Low Vision](#), Rubin and Legge, Vision Research 1989
- [The impact of using reverse polarity text for children with vision impairment assessed using light-adapted flicker electroretinogram](#), Loh et al., Clinical and Experimental Optometry, online 2024
- [Dark mode vogue: Do light-on-dark displays have measurable benefits to users?](#), Sethi and Ziat, Ergonomics 2023
- [Preference for Lighting Chromaticity in Migraine With Aura](#), Vieira et al., Headache 2020
- [Dark Mode or Light Mode? Exploring the Impact of Contrast Polarity on Visualization Performance Between Age Groups](#), While and Sarvghad, IEEE VIS 2024
- [The effect of coloured overlays and lenses on reading: a systematic review of the literature](#), Griffiths et al., Ophthalmic and Physiological Optics 2016
- [A systematic review of controlled trials on visual stress using Intuitive Overlays or the Intuitive Colorimeter](#), Evans and Allen, Journal of Optometry 2016

- [Developing Text Customisation Functionality Requirements of PDF Reader and Other User Agents](#), Henry, ICCHP 2012
- [Dyslexia friendly style guide](#), British Dyslexia Association, 2023, archived copy from July 2025 because the page has since moved
- [Understanding Success Criterion 1.4.8: Visual Presentation](#), W3C